Absolutely. This program implements **two stacks in a single array**.

The main idea is:

- **Stack 1 (S1)** grows from **left → right**
- **Stack 2 (S2)** grows from **right → left**
- Both stacks share the same array.
- The two stacks can use the available space efficiently.

For an array of size `8`, the indexes are:

```
Index:   0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
Array:  |    |    |    |    |    |    |    |    |
        +----+----+----+----+----+----+----+----+
```

---

1. Header File

```
C++

#include <iostream>
using namespace std;
```

`#include <iostream>`

This allows us to use:

- `cout` → display output
- `endl` → move to the next line

`using namespace std;`

So that we can write:

```
C++

cout
```

instead of:

```
C++

std::cout
```

---

2. Class Declaration

```cpp
class TwoStacks
{
```

We create a class named `TwoStacks`.

This class contains everything required to implement the two stacks.

---

3. Data Members

```cpp
int *arr;
int size;
int top1;
int top2;
```

There are four important variables.

`int *arr`

```cpp
int *arr;
```

This is a pointer that will point to the dynamically created array.

Later:

```cpp
arr = new int[size];
```

creates the array.

`int size`

```cpp
int size;
```

Stores the size of the array.

For:

```cpp
TwoStacks s(8);
```

we have:

```
size = 8
```

`int top1`

```cpp
int top1;
```

Keeps track of the **top element of Stack 1**.

Stack 1 starts from the left.

Initially:

```cpp
top1 = -1;
```

Why `-1`?

Because the array starts at index `0`, so `-1` means:

> Stack 1 currently has no element.

`int top2`

```cpp
int top2;
```

Keeps track of the **top element of Stack 2**.

Stack 2 starts from the right.

Initially:

```cpp
top2 = size;
```

For size `8`:

```
top2 = 8
```

Index `8` doesn't actually exist.

That's intentional.

It means:

> Stack 2 is currently empty.

---

4. Constructor

```cpp
TwoStacks(int n)
{
    size = n;
    arr = new int[size];

    top1 = -1;
    top2 = size;
}
```

The constructor executes when we write:

```cpp
TwoStacks s(8);
```

So:

```
n = 8
```

Step 1

C++

```
size = n;
```

Therefore:

```
size = 8
```

Step 2

C++

```
arr = new int[size];
```

Creates an array of 8 integers.

```
Index:   0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
Array:  |    |    |    |    |    |    |    |    |
        +----+----+----+----+----+----+----+----+
```

Step 3

C++

```
top1 = -1;
```

Stack 1 is empty.

```
top1
 ↓
-1
```

Step 4

C++

```
top2 = size;
```

Therefore:

```
top2 = 8
```

So initially:

```
top1 = -1
top2 = 8
```

---

5. `push1()` — Insert into Stack 1

```
C++

void push1(int value)
{
```

This function inserts an element into Stack 1.

Stack 1 grows:

```
LEFT → RIGHT
```

Overflow condition

```
C++

if (top1 + 1 == top2)
{
    cout << "Stack Overflow!" << endl;
    return;
}
```

This is the most important condition.

The two stacks are growing toward each other.

For example:

```
S1 → → →       ← ← ← S2
```

If the next position for Stack 1 is the same as the current position of Stack 2, there is no free space.

Therefore:

```
C++
```

```
top1 + 1 == top2
```

means the array is full.

Insert element

```C++
top1++;
arr[top1] = value;
```

First:

```C++
top1++;
```

Then:

```C++
arr[top1] = value;
```

For example:

```C++
s.push1(123);
```

Initially:

```
top1 = -1
```

After:

```C++
top1++;
```

we get:

```
top1 = 0
```

Then:

C++

```
arr[0] = 123;
```

Array becomes:

```
Index:   0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
Array:  |123 |    |    |    |    |    |    |    |
        +----+----+----+----+----+----+----+----+
          ↑
        top1
```

---

6. `push1(345)`

Now:

C++

```
s.push1(345);
```

Current:

```
top1 = 0
```

First:

C++

```
top1++;
```

So:

```
top1 = 1
```

Then:

C++

```
arr[1] = 345;
```

Array:

```
Index:   0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
Array:  |123 |345 |    |    |    |    |    |    |
        +----+----+----+----+----+----+----+----+
              ↑
             top1
```

---

7. `push1(456)`

C++

```
s.push1(456);
```

Again:

C++

```
top1++;
```

So:

```
top1 = 2
```

Then:

C++

```
arr[2] = 456;
```

Now:

```
Index:   0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
Array:  |123 |345 |456 |    |    |    |    |    |
        +----+----+----+----+----+----+----+----+
                   ↑
                  top1
```

Stack 1 is:

```
123
345
456 ← TOP
```

8. `push2()` — Insert into Stack 2

Now we insert into Stack 2:

```cpp
void push2(int value)
{
```

Stack 2 grows in the opposite direction:

```
RIGHT → LEFT
```

Overflow Check

```cpp
if (top1 + 1 == top2)
```

Same condition is used.

It checks whether both stacks have reached each other.

Move `top2`

```cpp
top2--;
```

This is important.

Initially:

```
top2 = 8
```

When we execute:

```cpp
s.push2(654);
```

we first do:

```cpp
top2--;
```

Therefore:

```
top2 = 7
```

Then:

```cpp
arr[top2] = value;
```

So:

```cpp
arr[7] = 654;
```

Array:

```
Index:   0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
Array:  |123 |345 |456 |    |    |    |    |654 |
        +----+----+----+----+----+----+----+----+
                                              ↑
                                             top2
```

Notice:

```
S1 → → →                ← ← ← S2
```

---

9. `push2(543)`

```cpp
s.push2(543);
```

Current:

```
top2 = 7
```

Execute:

```
C++
top2--;
```

Now:

```
top2 = 6
```

Then:

```
C++
arr[6] = 543;
```

Array:

```
Index:   0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
Array:  |123 |345 |456 |    |    |    |543 |654 |
        +----+----+----+----+----+----+----+----+
                                       ↑    ↑
                                      top2
```

---

10. `push2(321)`

```
C++
s.push2(321);
```

Current:

```
top2 = 6
```

Execute:

```
C++
top2--;
```

Now:

```
top2 = 5
```

Then:

```
    C++
arr[5] = 321;
```

Final array:

```
Index:   0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
Array:  |123 |345 |456 |    |321 |543 |654 |
        +----+----+----+----+----+----+----+----+
                   ↑          ↑              ↑
                  top1       top2
```

So the actual arrangement is:

```
S1                          S2
 ↓                           ↓
123  345  456  [empty]  321  543  654
 ↑                           ↑
Bottom                      Top
```

Remember that Stack 2 grows from **right to left**.

Therefore Stack 2 logically contains:

```
654
543
321
```

with:

```
654 = bottom
543
321 = top
```

---

11. `pop1()` — Remove from Stack 1

C++

```
int pop1()
{
```

This removes the top element from Stack 1.

First:

C++

```
if (top1 == -1)
```

checks whether Stack 1 is empty.

If:

```
top1 = -1
```

then there is nothing to remove.

Get the value

C++

```
int value = arr[top1];
```

Suppose:

```
top1 = 2
```

Then:

C++

```
value = arr[2];
```

which is:

```
456
```

Move top backward

C++

```
top1--;
```

So:

```
top1 = 1
```

Now Stack 1's top becomes:

```
345
```

Finally:

C++

```
return value;
```

returns:

```
456
```

---

12. `pop2()` — Remove from Stack 2

C++

```
int pop2()
{
```

First:

C++

```
if (top2 == size)
```

checks whether Stack 2 is empty.

Initially:

```
top2 = 8
```

Therefore Stack 2 is empty.

If it isn't empty, we do:

```cpp
int value = arr[top2];
```

Suppose:

```
top2 = 5
```

Then:

```
value = arr[5];
```

which is:

```
321
```

Then:

```cpp
top2++;
```

So:

```
top2 = 6
```

Therefore `321` is removed.

This follows **LIFO**:

```
Stack 2:

654  ← bottom
543
321  ← TOP

pop2()

321 is removed first
```

---

13. `display()`

C++

```
void display()
{
```

This function displays the complete array.

C++

```
cout << "\nArray: ";
```

prints:

```
Array:
```

Then:

C++

```
for (int i = 0; i < size; i++)
{
    cout << arr[i] << " ";
}
```

The loop runs from:

```
i = 0
```

to:

```
i = 7
```

For the current program, it prints:

```
123 345 456 [empty] 321 543 654
```

Important: Since the array was dynamically allocated using `new int[size]`, the unused position is **uninitialized**. So your program may print a garbage value there rather than an actual blank.

For example, you might see:

```
123 345 456 0 321 543 654  ...
```

or some other unpredictable integer.

If you want to display empty positions clearly, we should modify `display()`.

---

14. Display `top1`

C++

```
cout << "Top1 = " << top1 << endl;
```

After the three `push1()` operations:

```
top1 = 2
```

because:

```
arr[2] = 456
```

So:

```
Top1 = 2
```

---

15. Display `top2`

After three `push2()` operations:

```
top2 = 5
```

because:

```
arr[5] = 321
```

So:

```
Top2 = 5
```

---

16. `main()` Function

C++

```
int main()
{
    TwoStacks s(8);
```

This creates an object:

```
s
```

with an array of size:

```
8
```

So:

```
Index:  0   1   2   3   4   5   6   7
```

Initial values:

```
top1 = -1
top2 = 8
```

---

17. Insert Stack 1 Elements

C++

```
s.push1(123);
s.push1(345);
s.push1(456);
```

After these operations:

```
S1

Bottom
  ↓
123
345
456 ← TOP
```

Array:

```
Index:   0    1    2    3    4    5    6    7
        +----+----+----+----+----+----+----+----+
        |123 |345 |456 |    |    |    |    |    |
```

```
+----+----+----+----+----+----+----+----+
                ↑
               top1
```

---

18. Insert Stack 2 Elements

C++

```cpp
s.push2(654);
s.push2(543);
s.push2(321);
```

Because Stack 2 starts from the right:

First

```
654 → index 7
```

Second

```
543 → index 6
```

Third

```
321 → index 5
```

Final structure:

Here `???` represents **uninitialized positions**.

Actually, there are **two free positions**, indexes `3` and `4`.

---

19. Why This Implementation Is Efficient

Imagine we used two separate arrays:

```
Array 1 → Stack 1
Array 2 → Stack 2
```

We would have to divide memory beforehand.

For example:

```
Stack 1 → 4 positions
Stack 2 → 4 positions
```

But what if Stack 1 needs 6 positions and Stack 2 needs only 2?

The first stack could overflow even though the second stack has unused space.

With this implementation:

```
S1 → → →       ← ← ← S2
```

both stacks share the entire array.

Therefore, unused space belonging to one stack can be used by the other stack.

---

20. The Most Important Concept

Remember these four things:

```
Stack 1:

top1 = -1 initially

push1:
top1++
arr[top1] = value

pop1:
value = arr[top1]
top1--
```

And:

```
Stack 2:

top2 = size initially

push2:
top2--
arr[top2] = value

pop2:
value = arr[top2]
top2++
```

So the direction is:

```
                CENTER
                  ↓
S1:  → → → → →    ← ← ← ← ←  :S2
```

Actually, both stacks **start from opposite ends** and grow toward the center:

```
            S1 →       ← S2
            0  1  2  3  4  5  6  7
            ↑              ↑
          top1           top2
```

The stacks can continue growing until:

C++

```
top1 + 1 == top2
```

At that point:

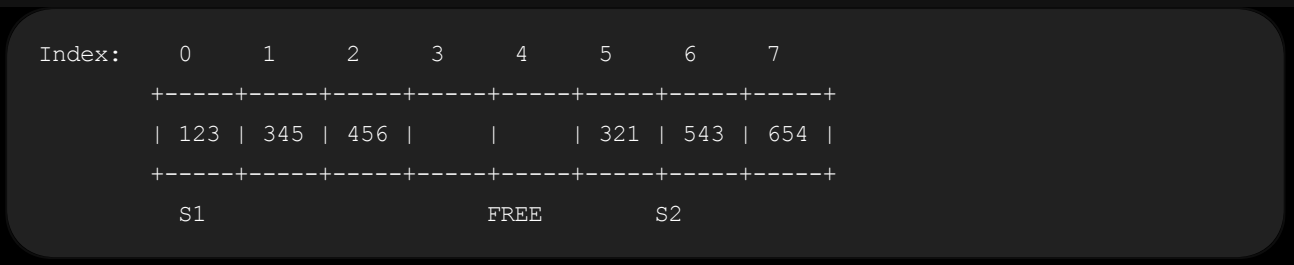
```
NO SPACE LEFT
      ↓
STACK OVERFLOW
```

Final state for your example

```
Stack 1:
Bottom → 123 → 345 → 456 ← Top

Stack 2:
Bottom → 654 → 543 → 321 ← Top
```

And in the shared array:



The key idea is simply: **S1 moves `top1` forward (`++`), while S2 moves `top2` backward (`--`).**